# MetroMind — Website Working & Demo Guide

A Complete Hindi/Hinglish Technical Walkthrough for Presentation & Viva

| | |
|---|---|
| **Project:** MetroMind (AI Spatio-Temporal Traffic System) | **Course:** E1CSA313 Design Project – I |
| **Team:** Akshat, Arya, Lucky, Sai Shikhar | **Mentor:** Dr. C. Viji · Alliance University |

## 0. Quick Presentation Cheatsheet (Sabse Pehle Kya Bolna Hai)

Kal jab evaluator ya Dr. Viji ke saamne khade hoge, toh seedha ye opening dialogue bolna:

> ***Opening Pitch (30 Seconds):***
> *'Good morning evaluators. Today we present MetroMind — an AI-driven spatio-temporal traffic optimization and bottleneck diagnosis framework for Bengaluru. Commercial navigation systems like Google Maps act like black boxes — they only show total ETA and often cause Braess's Paradox by diverting massive arterial volume onto fragile residential roads. MetroMind decomposes aggregate trip latency into section-level delays, diagnoses physical bottleneck causes, and enforces downstream spill-over constraints before recommending alternatives.'*

### 4 Golden Punchlines jo tumhein bolni hain:

**1. Graph Representation:** Humne Bengaluru ke 80 key junctions aur 17 main corridors ko ek weighted directed multigraph $G = (V, E)$ mein model kiya hai.

**2. Multi-Source TTI Fusion:** Travel Time Index calculate karne ke liye 65% live sensors feed (D3) aur 35% historical Kaggle machine learning baseline (D1) ko weather dampening coefficients ke sath blend kiya gaya hai.

**3. Spill-Over Check (Braess's Paradox):** Google Maps andha reroute karta hai, jabki MetroMind pehle check karta hai ki alternate raste par congestion **75% se kam** ho, taaki residential galiyon mein jam na lage.

**4. Privacy Floor (k-Anonymity):** User consent ke sath trip data cloud Neon Serverless PostgreSQL mein anonymous UUIDs ke sath save hota hai. Kisi user ka naam, phone ya exact location store nahi hoti.

---

## 1. Header & Live Clock Bar (Sabse Upar Ka Section)

### Kaunsa Feature Kya Karta Hai:

• **Live Clock (clk, dt):** Real Indian Standard Time (IST) aur current date dikhata hai. tick() function har 1 second mein isko update karta hai.

• **LIVE D3 Feed Indicator (lv, d3c):** Red pulsing dot aur live 5-minute slot count (jaise *096 / 288*). Poore 24 hours ko 5-5 min ke 288 simulation slots mein baanta gaya hai. sync() function har 5 second mein readings count update karta hai.

• **Pipeline Progress Bar (pbar):** 5 stages show karta hai: *1 Collect › 2 Preprocess › 3 Estimate › 4 Diagnose › 5 Guide*. Iske kisi bhi pill par click karne se page seedha us stage ke output par scroll hota hai.

• **Dataset Badges (D1–D5):** Header mein 5 dataset status pills hain jo system integrity show karte hain.

## 2. Hero Section & Trip Planner Form (User Input)

Commuter yahan apni travel parameters enter karta hai:

• **From (src) & To (dst) Autocomplete:** ac() function 80 Bengaluru places mein se instant suggestions deta hai. Agar koi aisi locality daali jo list mein nahi hai, toh online() function **OpenStreetMap Nominatim Geocoding API** se uske latitude/longitude nikal kar graph node se connect kar deta hai.

• **Swap Button (swap):** Ek click mein Source aur Destination ko interchange kar deta hai aur map redraw karta hai.

• **Departure Time (tm) & 'use live time' (now):** Commuter apna departure time set karta hai. 'Use live time' link dabane se system turant live IST time aur weekday/weekend status fill kar deta hai.

• **Travel Mode (mode):** Car, Two-wheeler, ya Bus select hota hai. adj() function vehicle density ke hisab se speed aur travel time mathematically adjust karta hai.

• **Weather (wx) & Day Type (day):** Clear, Rain, ya Heavy Rain. Barish hone par road friction badh jata hai aur speed drop calculate hoti hai.

• **Simulate Incident Checkbox (inc):** Artificial breakdown/accident simulate karta hai taaki evaluator ko dikha sako ki system incident wale road ko bottleneck kaise banata hai.

• **Analyse Trip Button (go):** Ye main trigger hai jo run(true) function execute karke computational pipeline start karta hai.

## 3. Pipeline Execution & Algorithmic Engine (Peeche Ka Dimag)

Jaise hi user 'Analyse trip' dabata hai, backend aur JS engine mein 4 technical processes chalte hain:

• **Modified Dijkstra Algorithm (path()):** Bengaluru ke 80-node aur 17-corridor road graph ko traverse karke dynamically lowest-latency primary path *R_prim* nikalta hai. Phir primary route par artificial penalty laga kar alternate route *R_alt* dhoondhta hai.

• **Travel Time Index Fusion (evalP(), ttiE()):** Har road section ka dynamic speed formula calculate hota hai: **Speed = Free_Speed / TTI**. TTI formula mein 65% weight live telemetry (D3) ko diya jata hai aur 35% weight historical Kaggle model (D1) ko.

• **Bottleneck Maximization:** Har road section ka delay delta *(tau_actual - tau_free)* evaluate karke critical choke point **e* = argmax(delay)** isolate hota hai.

• **Downstream Spill-Over Suppression (Cap <= 75%):** Alternate route ko tabhi suggest kiya jata hai agar uske kisi bhi secondary link par congestion 75% se kam rahe. Isse residential roads block nahi hote.

## 4. Interactive Live Map Section (Leaflet + Esri ArcGIS)

Map automatically smooth-scroll hokar display hota hai:

• **Esri ArcGIS HD Basemaps (initMap(), setBase()):** Leaflet 1.9.4 engine par Esri ArcGIS layers chalte hain. Upar 3 switch buttons hain: **Normal (World Street Map)**, **Night (Dark Canvas)**, aur **Satellite (World Imagery)**. Zero watermark aur zero OSM 403 errors.

• **Dynamic Congestion Polylines (drawMap()):** Route ke segments dynamic colors mein draw hote hain: ■ Green (<30% Free flow), ■ Yellow (30–54% Moderate), ■ Orange (55–74% Heavy), ■ Red (75%+ Severe).

• **Dashed Purple Polyline:** Alternate candidate route ko map par visualize karti hai.

• **Pulsing Bottleneck Marker (bnm):** Jo road section delay create kar raha hai, wahan red pulsing beacon blink karta hai.

• **Animated Vehicle Tracers (veh):** 3 yellow dots route ke upar lagatar smooth animate hote hain jo traffic velocity ka live visual feed dete hain.

• **Map Tools:** 'Fit route' map ko bounding box par zoom karta hai, aur 'Full city' poore Bengaluru ka wide view reset karta hai.

## 5. Route Options Cards (Route 1 vs Route 2 Picker)

• **Function ropts() & selRoute():** Jab alternate route available hota hai, toh map ke neeche do interactive cards bante hain: **Route 1** aur **Route 2**.

• **Smart Tags:** Cards par *Fastest*, *Shortest*, ya *Least Congested* ke labels aate hain.

• **One-Click Switch:** User kisi bhi card par click karke map polyline, bottleneck aur KPIs ko instant switch karke dekh sakta hai.

## 6. Diagnostic KPIs, Bottleneck Attribution & What-If Engine

• **Animated KPI Cards (kpis):** 6 metric cards animated count-up ke sath aate hain: Estimated Travel Time (min), Total Distance (km), Average Speed (km/h), Delay vs Free Flow (+min), Congestion Index (%), aur Estimated Arrival Time.

• **Causal Bottleneck Banner (bn):** Na sirf bottleneck section batata hai (e.g., *HSR Layout → Singasandra*), balki physical cause bhi explain karta hai: whether peak vehicle-to-capacity volume, weather drag, ya incident report.

• **Spill-Over Verified Guidance (sg):** Commuter ko high-value decision suggest karta hai: jaise **'Leave 90 min later (10:00) — Saves 18 min'**. Sath mein ek button hota hai: **'Apply this time'** jo form ka time automatic set kar deta hai.

• **Section-wise Breakdown Table (secs):** Har individual road segment ki length, speed, arrival timestamp aur delay bar tabular format mein aati hai.

• **What-If Comparison Table (cmp):** Current plan ko departure time shifts (±15 to ±90 min), route changes, aur modal switches ke sath compare karta hai.

## 7. Trip Save & Database Persistence (Neon Cloud PostgreSQL)

• **Consent Checkbox (cons):** Commuter privacy ke liye explicit consent checkbox hai: *'I consent to adding this trip to the anonymous corridor summary'*. Check karne par hi 'Save trip (D2)' button active hota hai.

• **Asynchronous API Save ($('save').onclick):** Ye function Express backend ko JSON payload dispatch karta hai: POST /api/trips.

• **Cloud Neon PostgreSQL Storage:** Express backend cloud-hosted **Neon Serverless PostgreSQL (AWS us-east-2)** database ke trips table mein record persist karta hai.

• **Differential Privacy:** Database mein koi user name, mobile number ya exact GPS trajectory store nahi hoti. Sirf anonymous UUIDs generate hoti hain (e.g. T-0414, U-012) aur section-level identifiers persist hote hain.

## 8. Corridor Dashboard Tab (Macro Transit Intelligence)

Header mein **'Corridor Dashboard'** tab click karne par macro analytics khulte hain:

• **Hourly City Congestion Profile (cpc, cityProfile()):** Custom SVG bar chart jo 24 ghante ka average congestion level show karta hai. Subah 08:30 (87%) aur shaam 18:30 (86%) peak hours red mein highlighted hote hain, aur dopahar 28% green mein hoti hai. Upar Weekday / Weekend toggle button hai.

• **Corridor Ranking Slider (rank()):** Ek dynamic time slider (05:00 to 22:00) hai. Slider ko drag karte hi Bengaluru ke sabhi 17 corridors real-time mein most congested se least congested sort ho jate hain.

• **Anonymous Corridor Delay Heatmap (heat):** 17 corridors x 16 hours ka delay matrix grid. **k-Anonymity Safeguard:** Jis cell mein 5 se kam trips hoti hain, wahan cell mask rehti hai (—), taaki kisi individual commuter ki identity leak na ho sake.

• **Saved Trips Session Table (trips):** Jo trip user ne Trip Planner mein save ki thi, wo yahan anonymous ID ke sath display hoti hai.

## 9. Datasets & Pipeline Tab (Academic Verification)

• **Interactive Walkthrough (wireX(), stage()):** 5 buttons hain: Collect, Preprocess, Estimate, Diagnose, Guide. **'■ Play walkthrough'** button dabane se pipeline automated play hoti hai aur har stage ka dataset input aur output highlight karti hai.

• **Architecture Safeguards Cards:** 65/35 Live-Historical Blend, 75% Spill-over limit, aur k >= 5 privacy floor ke architectural cards yahan verified hain.

• **Dataset Summary:** D1 (Historical Kaggle, 16,705 records), D2 (Commuter probe trips), D3 (Live 5-min feeds), D4 (Road graph & weather), aur D5 (Anonymous aggregate summary).

## 10. Expected Evaluator Viva Questions & Ready Answers

| Evaluator Ka Question | Tumhara Exact Technical Jawab |
|---|---|
| **Q1:** Travel time calculation ka mathematical formula kya hai? | **Ans:** Section speed formula hota hai *v_e = v_free / TTI_e*. Travel Time Index (TTI) calculate karne ke liye hum 65% weight live telemetry (D3) ko dete hain aur 35% weight Kaggle historical baseline (D1) ko, jo weather dampening coefficient se multiply hota hai. |
| **Q2:** Google Maps se tumhara system alag kaise hai? | **Ans:** Google Maps individual car ko greedy shortest path deta hai jisse residential colonies mein traffic jam lagta hai (Braess's Paradox). MetroMind ek **Downstream Spill-Over Constraint** lagata hai: alternate route tabhi suggest hoga agar secondary roads par congestion 75% se kam ho. |
| **Q3:** Database aur backend architecture kya hai? | **Ans:** Full-stack decoupled architecture hai: Frontend Vanilla ES6+ aur Leaflet 1.9.4 with Esri ArcGIS maps, Backend Node.js Express REST API (/api/trips), aur Database cloud-hosted **Neon Serverless PostgreSQL (AWS us-east-2)** with SSL connection pooling. |
| **Q4:** User privacy kaise preserve ho rahi hai? | **Ans:** System differential k-anonymity (k >= 5) follow karta hai. No names, phone numbers, ya continuous GPS breadcrumbs are stored. Sirf random UUIDs (T-xxxx, U-xxxx) aur road section segments persist hote hain. |